# CS223 Computer Architecture & Organization

## Cache Memory – Indexing


**John Jose**

**Associate Professor**

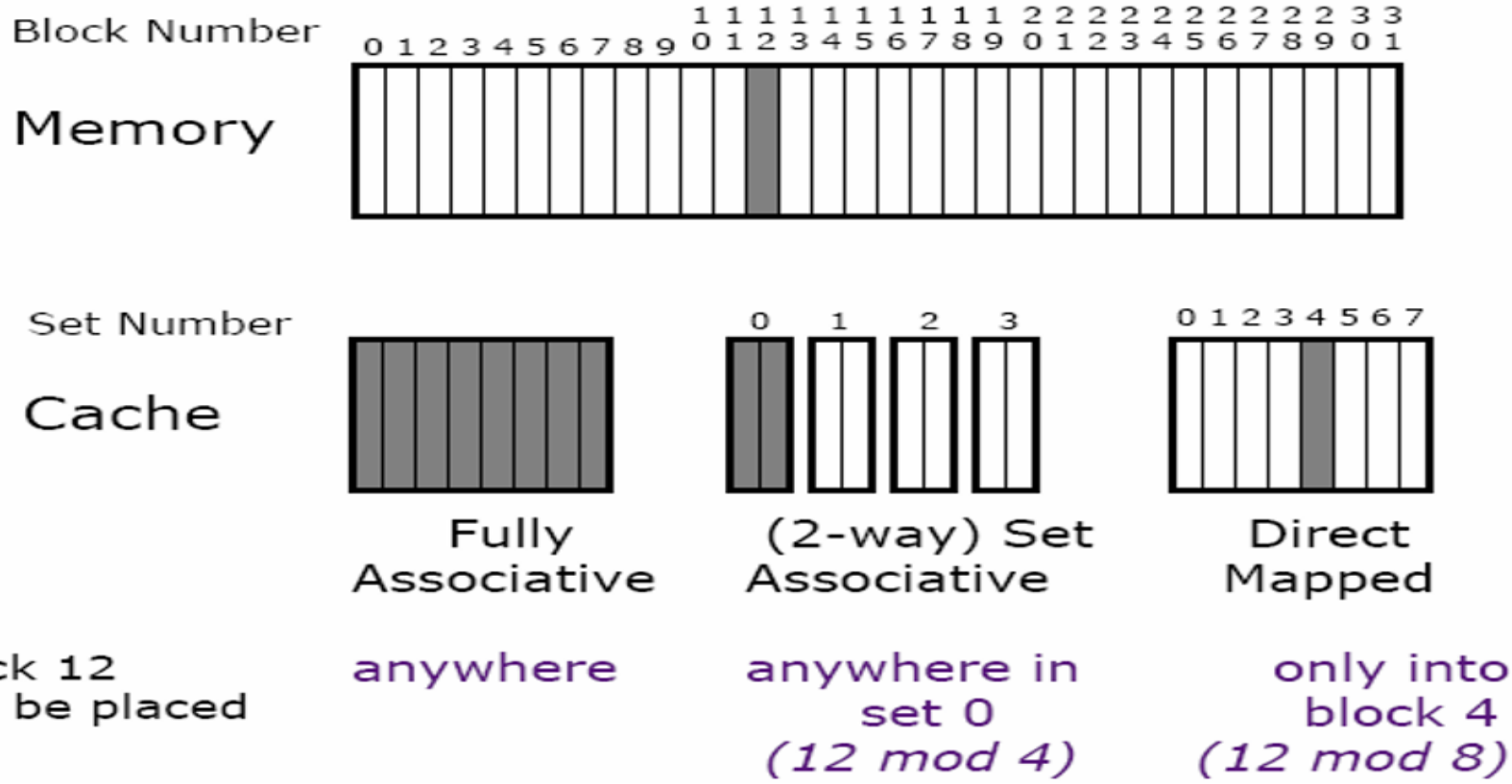
**Department of Computer Science & Engineering**

**Indian Institute of Technology Guwahati**

# Four cache memory design choices

- Where can a block be placed in the cache?
  - **Block Placement**
- How is a block found if it is in the upper level?
  - **Block Identification**
- Which block should be replaced on a miss?
  - **Block Replacement**
- What happens on a write?
  - **Write Strategy**

# Block Placement



# Index and Offset Calculations

A cache has 512 KB capacity, 4B word, 64B block size and 8-way set associative. The system is using 32 bit address. Given the address 0X ABC89984, which set of cache will be searched and specify which word of the selected cache block will be forwarded if it is a hit in cache?

# sets = CS/(BSxA) = $2^{19}/(2^6 x 2^3) = 2^{10}$ = 1024 sets

1 word = 4B , Hence 64 byte block has 16 words

| Tag = 16 | Index =10 | Offset=6 (4+2) |
|---|---|---|

0x ABC89984 = 1001 1001 1000 0100 → Set 614, word 1

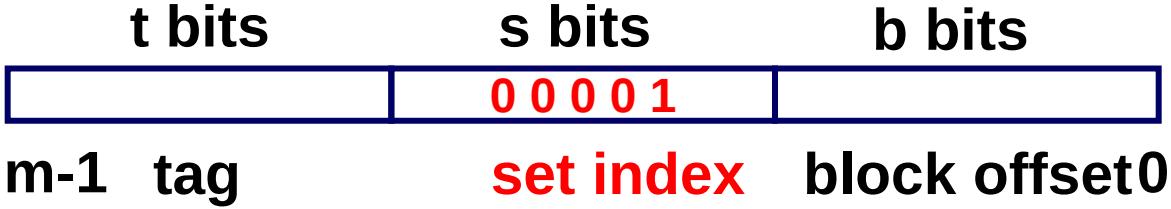
0x 485669AC = 0110 1001 1010 1100 → Set 422, word 11

# Mapping Calculations

The address of a word in a byte addressable 1MB physical memory is 0xAB8F2. This word upon bringing to the cache is mapped to set 30. The word length of the processor is 16 bits. How many words can be accommodated in each cache block?

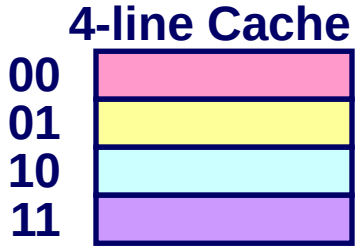
- 0xAB8F2→ 10101011100011110010
- 30→ 11110: We can see the set index 30 (11110) above. RHS of set index is byte offset.
- offset → 010→ 8 bytes. one word is 2 bytes [16 bits] therefore 8 byte offset can accommodate 4 words

# Cache Indexing



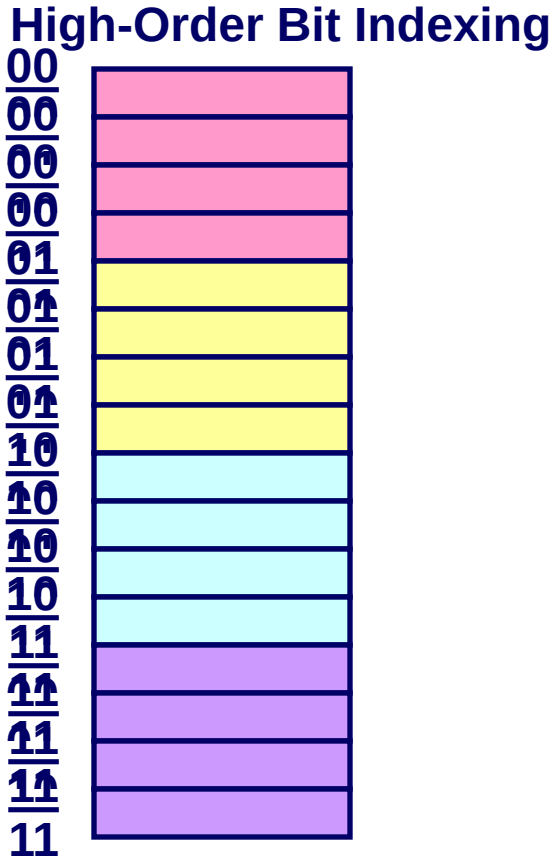

- Decoders are used for indexing
- Indexing time depends on decoder size ( s: $2^s$)
- Smaller number of sets, less indexing time.

# Why Use Middle Bits as Index?

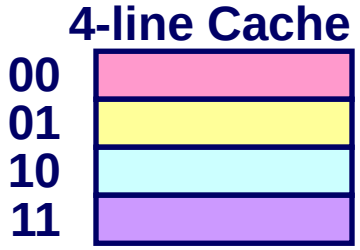



## High-Order Bit Indexing

- Adjacent memory lines would map to same cache entry
- Poor use of spatial locality

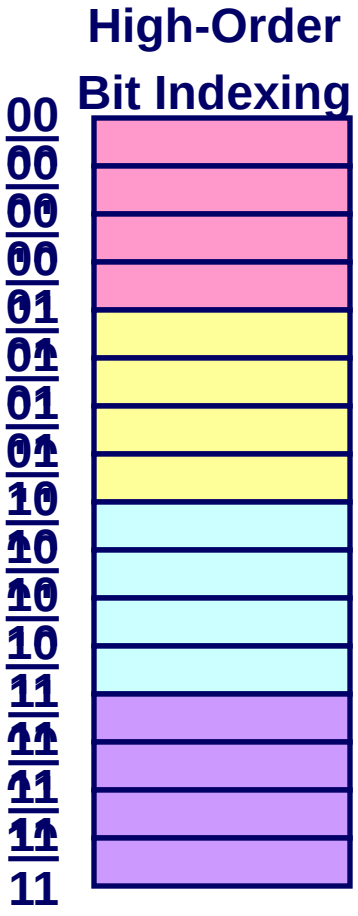


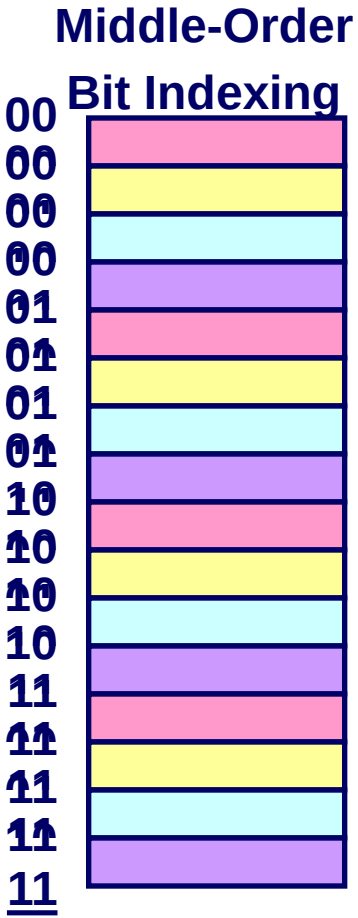
# Why Use Middle Bits as Index?




## Middle-Order Bit Indexing

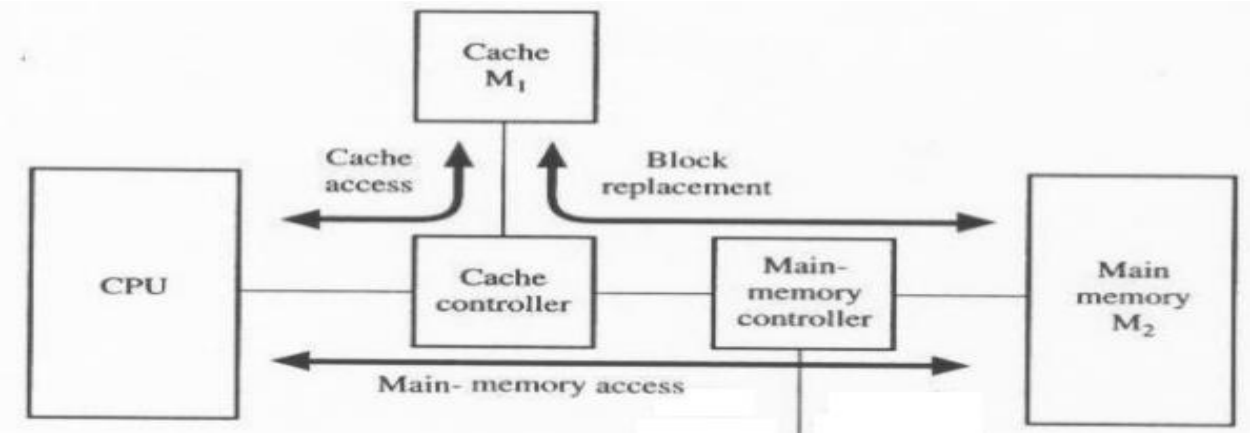
- Consecutive memory lines map to different cache lines
- Better use of spacial locality without replacement



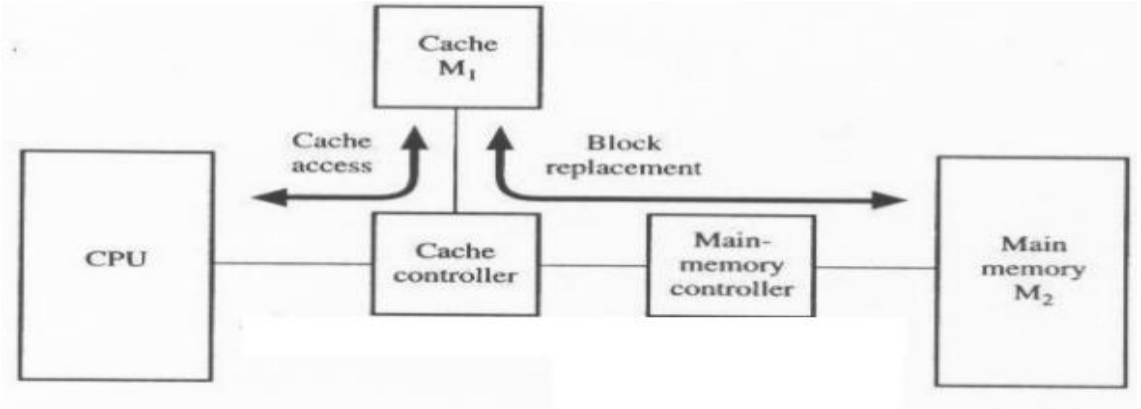
# Look-aside vs Look through caches

- **Look-aside cache:** Request from processor goes to cache and main memory in parallel
- Cache and main memory both see the bus cycle
- On cache hit→processor loaded from cache, bus cycle terminates; On cache miss: processor & cache loaded from memory in parallel



# Look-aside vs Look through caches

- **Look-through cache:** Cache checked first when processor requests data from memory
- On hit☐ data loaded from cache: On miss☐ cache loaded from memory, then processor loaded from cache




**johnjose@iitg.ac.in**
**http://www.iitg.ac.in/johnjose/**